Dynamo-Inspired Fault-Tolerant Key-Value Store

Team Number: 18

Team members: Dhruv Rajput - 2025201094, Sohail Memon - 2025201075

---

1. Project Description

We will build a key-value store inspired by Amazon's Dynamo. It runs as a cluster of **4 to 6 processes**. Clients use a simple **`GET` / `PUT` API**.

The main parts of the system are:

- **Partitioning and replicas.** Keys are placed on nodes using **consistent hashing**. Every key is stored on **at least three nodes**, so the data is still available if one node fails.
- **Failure detection.** Nodes send **heartbeat** messages to each other. A node that misses a fixed number of heartbeats is marked as unavailable.
- **Failure tolerance.** The system keeps handling `GET` and `PUT` requests when a node is down. We will state the failure tolerance clearly: with 3 copies, writes need 2 acknowledgements and reads use 2 replicas, so the system survives **1 node failure**.
- **Durable versions.** Each node saves every version of a key on disk, so the data is not lost when a process restarts.
- **Repair (anti-entropy).** When a node comes back, it compares **digests** (hashes) of its data with the other replicas. The digests show which updates are missing or old, and the node then fetches the latest versions.
- **No lost writes.** We will show that, under our chosen write policy, a write that was acknowledged to the client is never lost, even after a failure and recovery.

To test this, we will **kill a replica while clients are writing, restart it**, and measure the **repair time** and the **request availability** during the failure.

The project covers these ideas: consistent hashing, replication, failure detection, anti-entropy, and durability.

2. Goals

1. **Build the cluster and API**
   - Run a cluster of 4 to 6 server processes.
   - Support `PUT(key, value)` and `GET(key)`.
   - Return a clear success or error to the client for every request.
2. **Partition data with consistent hashing**
   - Place nodes and keys on a hash ring.
   - Use virtual nodes so keys are spread evenly across nodes.
   - Find the nodes for a key by walking clockwise on the ring.
3. **Keep multiple copies of every key**
   - Store each key on at least **three different nodes** (N = 3).
   - Choose a write policy: a `PUT` is acknowledged only after **2 replicas** have saved it (W = 2).
   - Read from **2 replicas** (R = 2) and return the newest version.
4. **Detect unavailable nodes**
   - Send heartbeat messages between nodes at a fixed interval (for example, every 1 second).
   - Mark a node as down after a fixed number of missed heartbeats (for example, 3).
   - Mark the node as up again when its heartbeats return.
   - Measure how long it takes to detect a failure.
5. **Continue working within a stated failure tolerance**
   - State clearly that the system survives **1 node failure**.
   - Keep serving `GET` and `PUT` requests while one node is down.
   - Explain what happens if more nodes fail (some requests may fail).
6. **Store versions durably**
   - Write every version of a key to a log file on disk.
   - Save the data to disk before sending the acknowledgement to the client.
   - Reload all data from disk when a node restarts.
7. **Repair missing or stale replicas (anti-entropy)**
   - Let each node build digests (hashes) of its data.
   - When a node returns, compare its digests with the other replicas.
   - Use the digests to find keys that are missing or old.

- Copy only those keys from a healthy replica.

8. **Show that no acknowledged write is lost**
   - Have the client record every write that was acknowledged.
   - After a failure and recovery, read all those keys back.
   - Check that every acknowledged write is still there.
9. **Test and measure failure and recovery**
   - Kill a replica while clients are writing, then restart it.
   - Measure the **repair time** (restart until all data is up to date).
   - Measure the **request availability** (share of requests that succeed) during the failure.
   - Repeat the test with different loads and numbers of keys.

3. Tools and Technologies

Item	Choice
Programming language	C++ or Go
Communication	gRPC with Protocol Buffers
Node processes	Separate processes, one per node
Durable storage	Append-only log file per node (with `fsync`), optionally with an embedded store such as RocksDB / LevelDB
Hashing	MD5 / SHA-1 for the hash ring and for replica digests

4. Special System Requirements

- We will build, run, and test the project on our **RCE accounts**, which give us **8 nodes** in total.
- Using separate machines gives more realistic failure and repair timings than running everything on one machine.
- Each node needs local disk space for its log files and stored versions.
- We need permission to kill and restart processes on the RCE nodes.
- **Storage:** each server node has its own **local disk**, not a shared network disk (such as NFS). With a shared disk, replicas would not really be separate, and `fsync` timings would not be realistic.
- **Clocks:** node clocks should be in sync (NTP), so we can measure repair time correctly.
- **Software:** a C++ compiler or Go toolchain, with gRPC and Protocol Buffers installed (or installable in our home directory).

5. Expected Outcomes

Working system

- A running cluster with consistent-hash partitioning and three replicas per key.
- A `GET` / `PUT` API with a clear write policy (2 of 3 replicas must save a write before it is acknowledged).
- Heartbeat-based failure detection.
- Durable storage of key versions.
- Digest-based anti-entropy repair for nodes that return after a failure.

Main experiment

We will run a client workload that keeps sending `PUT` and `GET` requests. While it runs, we will:

1. Kill one replica during the writes.
2. Keep the workload running while the node is down.
3. Restart the node.
4. Measure the results.

We will report:

- **Request availability:** the share of requests that succeed while the node is down.
- **Repair time:** the time from restart until the node has all the latest data.
- **Data safety check:** every `PUT` that was acknowledged can still be read after recovery. No acknowledged write is lost.

We will also repeat the test with different load levels and different numbers of keys, to see how they change repair time.

Deliverables

- Source code with a README that explains how to run the cluster and the tests.
- Scripts for the kill-and-restart experiment.
- Graphs and tables with the results.